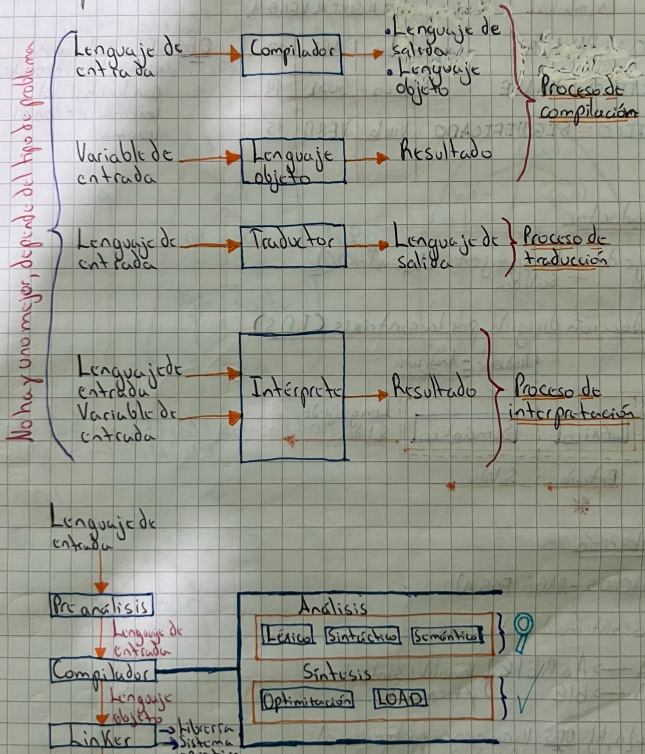
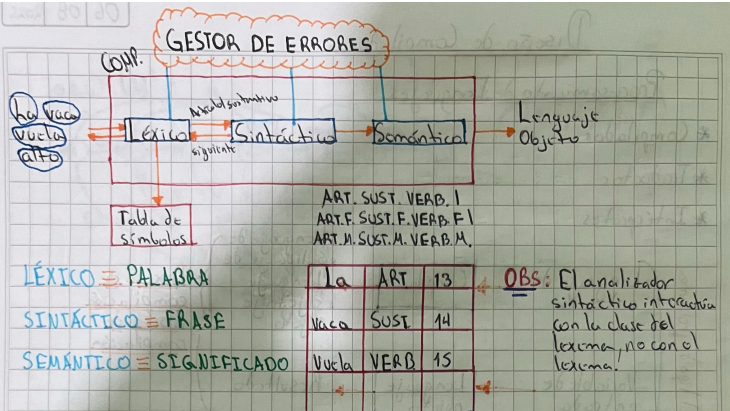
06 08 2026

Diseño de Compiladores

Procesamiento de Lenguajes

* Compilados
* Traductor
* Intérpretes





LÉXICO ≡ PALABRA

SINTÁCTICO ≡ FRASE

SEMÁNTICO ≡ SIGNIFICADO

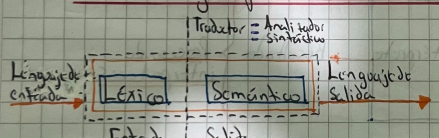
La	ART	13
Vaca	SUST	14
Vuela	VERB	15

OBS: El analizador sintáctico interactúa con la clase del lexema, no con el lexema.

Traductor

Lenguaje de entrada = Lenguaje de salida (Equivalencia)

Traducción dirigida por la sintáxis (T.D.S)



Notación

BNF (BACKUS - NAUR FORM)

Jerarquía: sistema de producciones.

Variables sintácticas $\begin{cases} A \rightarrow AaBb|\varepsilon \\ B \rightarrow b|\varepsilon \end{cases}$ Variables sintácticas y terminales.

$\Sigma = \{a, b\}$ OBS: Vacío nunca pertenece al alfabeto de entrada.

$a \rightarrow AaBb|\varepsilon$

$b \rightarrow B|\varepsilon$

$\Sigma = \{A, B\}$

$[cad \rightarrow cad+cad \mid cad-cad \mid digito]$

$digito \rightarrow 0|1|2|...|9$

$\Sigma = \{+, -, 0, 1, 2, ..., 9\}$

Árbol Sintáctico

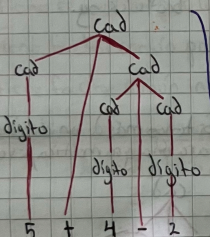
padre hijos

A → x y z

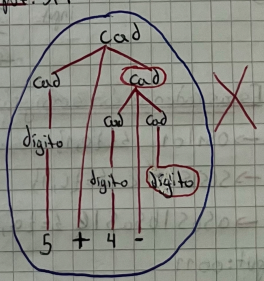
= raíz/nodos de entrada ≡ variable sintáctica
= hojas ≡ terminales
raíz ≡ variable sintáctica inicial

[cad → cad + cad | cad - cad | dígito]
dígito → 0|1|2|...|9
Σ = {+, -, 0, 1, 2, ..., 9}

input: 5+4-2



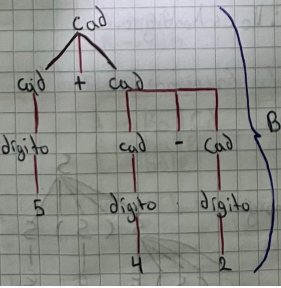
input: 5+4-



[cad → cad + cad | cad - cad | dígito]
dígito → 0|1|2|...|9

input: 5+4-2

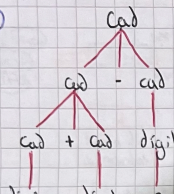
①



Left most (Lm)

cad ⇒ cad + cad ⇒ dígito + cad ⇒
5 + cad ⇒ 5 + cad - cad ⇒
5 + dígito - cad ⇒ 5 + 4 - cad ⇒
5 + 4 - dígito ⇒ 5 + 4 - 2

②



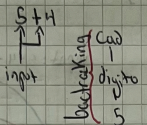
Right most (Rm)

$cad \Rightarrow cad - cad \Rightarrow cad - digito \Rightarrow$

$cad - 2 \Rightarrow cad + cad - 2 \Rightarrow$

$cad + digito - 2 \Rightarrow cad + 4 - 2 \Rightarrow$

$digito + 4 - 2 \Rightarrow 5 + 4 - 2$



Obs:
- Siempre vamos a ir de izquierda a derecha.
- Hay que evitar siempre el backtracking.

Estas cadenas son ambiguas?

a) $S \rightarrow 0S1 | 01 \quad \Sigma = \{0,1\}$

b) $S \rightarrow S(S)S | \varepsilon \quad \Sigma = \{(,)\}$

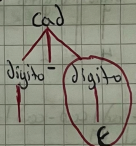
c) $S \rightarrow aSbS | bSaS | \varepsilon \quad \Sigma = \{a,b\}$

→ input: 0011

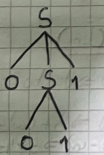
→ input: ()()

→ input: abab

Obs: Si tengo más de un árbol sintáctico distinto para una gramática, es ambigua.



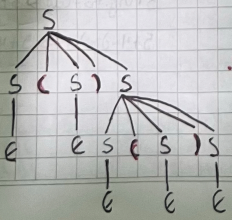
a)



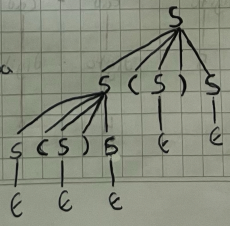
No hay dos árbol sintactico distinto para una sola entrada.

∴ No es Ambigua

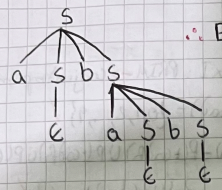
b)



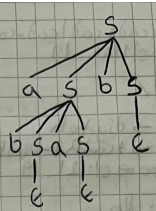
∴ Es Ambigua



c)



∴ Es Ambigüa



Una gramática es predictiva ssi:

1- No tiene recursión por la izquierda.

2- No tiene factor común por la izquierda.

3- No tiene factor común indirecto por la izquierda.

Cómo eliminar la recursión por la izquierda

$$A \to A\alpha \mid \beta$$

$$\equiv$$

$$A \to \beta R$$
$$R \to \alpha R \mid \varepsilon$$

Cómo eliminar factor común por la izquierda

$$A \to \beta\alpha \mid \beta b$$

$$\equiv$$

$$A \to \beta R$$
$$R \to \alpha \mid b$$

Ejemplo de factor común indirecto por la izquierda

$$A \to Ba \mid Cd$$
$$B \to ba \mid \varepsilon$$
$$C \to bd \mid \varepsilon$$

OBS:

- Eliminar primero la recursión por la izquierda facilita mucho más el trabajo porque resuelve el problema estructural más grave de los analizadores top-down y despeja el camino para que la factorización sea rápida y directa.
- Hacer siempre de arriba hacia abajo para la solución del factor común indirecto.
- Realizar siempre la unión para no confundirse.
- No hay factor común indirecto si todas las uniones son disjuntas.
- El conjunto primero puede fallar pero todavía no veremos eso.

[cad → cad + cad | cad - cad | digito
digito → 0|1|2|...|9]

① R.x.I: ✓

cad → digito R
R → + cad R | - cad R | ε
digito → 0|1|2|...|9

② F.C.x I.: No

③ F.C.I x I.: PRIM → conjunto primero

P(cad) = {0,1,2,...,9} ✓

P(R) = P(+ cad R) ∪ P(- cad R) ∪ P(ε) = {+} ∪ {-} ∪ {ε} = {+,-,ε} ✓

P(digito) = {0} ∪ {1} ∪ {2} ∪ ... ∪ {9} = {0,1,2,...,9} ✓

Obs: • Variable sintáctica → función
• Variable terminal → match

```
cad(){
  digito(); R();
}
```

```
R(){
  if(input == '+'){
    match('+');
    cad();
    R();
  }else if(input == '-'){
    match('-');
    cad();
    R();
  }else
    ;
}
```

```
digito(){
  if(input == '0'){
    match('0');
  }else if(input == '1'){
    match('1');
  }else if(input == '2')
    match('2');
  }
  .
  .
  .
  else{
    match('9');
  }
```

```
match(t){
  if(input == t){
    input = get next input();
  }else
    error();
  }
```

Ejemplo de error: 0+-7

Verificar si estas cadenas son predictivas

a) S → 0S1|01 Σ = {0,1}

b) S → S(S)S | ε Σ = {(,)}

c) S → aSbS | bSaS | ε Σ = {a,b}

a) $S \to 0S1 \mid 01$

① F.x.I.: No ② F.C.xI.: Si ③ F.C.IxI.: PRIM

$S \to 0R$

$R \to S1 \mid 1$

$P(S) = \{0\}$ ✓

$P(R) = P(S1) \cup P(1) = \{0\} \cup \{1\} = \{0, 1\}$ ✓

```
S(){
  if (input == '0'){
    match('0');
    R();
  }
}
```

```
R(){
  if (input == '0'){
    S();
    match('1');
  }else
    match('1');
}
```

```
match(t){
  if (input == t)
    input = getNextInput();
  else
    error;
}
```

OBS. • Solo se agrega ϵ al factorizar si una de las opciones originales consistía únicamente en el factor común (sin nada más pegado después). Ejemplo:

$A \to \delta c \mid \delta$ $\quad \delta.(c \mid \text{"nada"})$

$A \to \delta R$

$R \to c \mid \epsilon$

• Los conjuntos primeros solo contienen símbolos terminales. Ejemplo:

$P(S1) = P(S)$, como $P(S) = \{0\}$, automáticamente:

$P(S1) = \{0\}$

• ¿Cuándo habría importado el 1 de S1?: El 1 que está al lado de la S solo habría formado parte del conjunto primero si la S pudiera derivar en la cadena vacía (ϵ). Como S no es anulable (es decir, $S \neq \epsilon$ porque siempre empieza obligatoriamente con 0), la S actúa como un "muro", debido a la búsqueda ahí mismo y el 1 que está después nunca podrá ser el primer carácter de la cadena.

• El conjunto Primero te indica exacto qué tokens de entrada "desencadenan" la ejecución de cada regla de producción.

b) $S \to \$(S)S \mid \$ \mid \epsilon$

① R.x I.: Si ② F.C.I x I: NO ③ F.C.I x I.: PRIM

$S \to R$
$R \to (\$)SR \mid \epsilon$

$P(S) = P(R) = \{\epsilon\}$ ✓
$P(R) = P(\$)SR) \cup P(\epsilon) = \{\$\}$
$\cup \{\epsilon\} = \{\$, \epsilon\}$ ✓

```
S(){
    R();
}

R(){
    if(input=='$'){
        match('$');
        S();
        match('(');
        S();
        match(')');
        S();
        R();
    }else{
        return;
    }
}

match(t){
    if(input==t){
        input=getnextinput();
    }else{
        error();
    }
}
```

c) $S \to aSbS \mid bSaS \mid \epsilon$

① R.x I.: No ② F.C.I x I.: No ③ F.C.I x I.: PRIM

$P(S) = P(aSbS) \cup P(bSaS) \cup P(\epsilon) =$

$\{a\} \cup \{b\} \cup \{\epsilon\} = \{a, b, \epsilon\}$

```
S(){
    if(input=='a'){
        match('a');
        S();
        match('b');
        S();
    }else if(input=='b'){
        match('b');
        S();
        match('a');
        S();
    }else{
        return;
    }
}

match(t){
    if(input==t){
        input=getnextinput();
    }else{
        error();
    }
}
```